

FULL STACK WEB DEVELOPMENT

USE CASE - ASSIGNMENT

Subject Code: 21CSE354T

STUDENT NAME	S.Sam Glad Priyan
REGISTER NUMBER	RA2411003050138
PROGRAM / SECTION	Computer Science Engineering / "C" Section
YEAR / SEMESTER	III-Year / V-Semester
ASSIGNMENT / USE CASE NO.	Sphere volume calculation / No.20
DATE OF SUBMISSION	31/08/2026

COURSE	Full Stack Web Development
FACULTY COORDINATOR	Dr. P. Hariharan
ACADEMIC YEAR	2024 - 2028

"Design * Develop * Deploy"

1. Introduction

This report outlines the design and implementation of a simple, modern Sphere Volume Calculator webpage. The project has been developed as a single-page frontend application, strictly conforming to basic course assignment requirements for simplicity, clarity, and modularity. All structural markup, styling, and interactivity are bundled in a single file to allow ease of testing and deployment.

2. Key Features

- **Input Numeric Validation:** The application checks the user's input, ensuring that only positive numerical values can be submitted for the sphere's radius.
- **Interactive Calculation Trigger:** On clicking the 'Calculate' button, the volume computation function is called to perform mathematical calculations in real time.
- **Formula Implementation:** Computes the exact volume of the sphere using the standard formula $V = (4/3) * \pi * r^3$, formatted to 4 decimal places for scientific precision.
- **ReadOnly Results Box:** The calculated output is presented inside a second text box set to read-only mode, securing result integrity.

3. Architecture & Implementation

HTML Structure

The HTML layout uses a card wrapper container (.calculator-card). It contains form label blocks, a numeric input field for the radius, a text input field for the output volume, and a triggering button. Semantic structure ensures screen readability and SEO compatibility.

CSS Design & Styling

The CSS styling provides a modern user interface card component, centered perfectly in the viewport using Flexbox. Features include soft drop shadows, rounded input borders, and responsive scaling. Active states on buttons and focus rings on text fields offer immediate tactile user feedback.

JavaScript Program Flow

The interactive script triggers upon clicking the 'Calculate' button. The logic retrieves the radius value, parses it, validates it against empty or negative numbers, applies the mathematical formula using Math.PI and Math.pow, and updates the DOM textContent of the output read-only box.

4. Source Code (index.html)

The code below represents the complete implementation of the webpage:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Sphere Volume Calculator</title>
<style>
  /* General page layout */
  body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f3f4f6;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
  }

  /* Card layout */
  .calculator-card {
    background-color: #ffffff;
    padding: 35px 30px;
    border-radius: 16px;
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
    width: 320px;
  }

  /* Title style */
  h1 {
    margin: 0 0 5px 0;
    color: #1f2937;
    font-size: 22px;
    text-align: center;
  }

  p {
    margin: 0 0 20px 0;
    color: #6b7280;
    font-size: 13px;
    text-align: center;
  }

  /* Input field container */
  .input-group {
    margin-bottom: 20px;
  }

  label {
    display: block;
    margin-bottom: 6px;
    font-size: 14px;
    font-weight: 600;
    color: #4b5563;
  }

  input {
    width: 100%;
    padding: 10px 12px;
    font-size: 15px;
    border: 1px solid #d1d5db;
    border-radius: 8px;
    box-sizing: border-box;
    outline: none;
    transition: border-color 0.2s;
  }

  input:focus {
```

```

        border-color: #2563eb;
    }

    /* Styling for readonly output box */
    input[readonly] {
        background-color: #f9fafb;
        color: #374151;
        font-weight: 600;
    }

    /* Button style */
    button {
        width: 100%;
        padding: 12px;
        font-size: 16px;
        font-weight: 600;
        background-color: #2563eb;
        color: #ffffff;
        border: none;
        border-radius: 8px;
        cursor: pointer;
        transition: background-color 0.2s;
    }

    button:hover {
        background-color: #1d4ed8;
    }

    /* Error message style */
    .error-message {
        color: #ef4444;
        font-size: 12px;
        margin-top: 5px;
        display: none;
    }
}
</style>
</head>
<body>

    <div class="calculator-card">
        <h1>Volume of a Sphere</h1>
        <p>Enter the radius to calculate the volume</p>

        <!-- Input 1: Radius -->
        <div class="input-group">
            <label for="radius">Radius (r):</label>
            <input type="number" id="radius" step="any" placeholder="Enter radius" min="0">
            <div id="error" class="error-message">Please enter a valid positive number</div>
        </div>

        <!-- Input 2: Output Volume -->
        <div class="input-group">
            <label for="volume">Volume (V):</label>
            <input type="text" id="volume" placeholder="Calculated volume output" readonly>
        </div>

        <!-- Calculate Button -->
        <button id="calculateBtn">Calculate</button>
    </div>

    <script>
        // DOM element references

```

```
const radiusInput = document.getElementById('radius');
const volumeInput = document.getElementById('volume');
const calculateBtn = document.getElementById('calculateBtn');
const errorDiv = document.getElementById('error');

// Volume calculation handler
function calculateVolume() {
  const radius = parseFloat(radiusInput.value);

  // Check for valid positive numerical input
  if (isNaN(radius) || radius < 0) {
    errorDiv.style.display = 'block';
    volumeInput.value = '';
    return;
  }

  // Hide error message on successful calculation
  errorDiv.style.display = 'none';

  // Formula:  $V = \frac{4}{3} * \pi * r^3$ 
  const volume = (4 / 3) * Math.PI * Math.pow(radius, 3);

  // Output formatted to 4 decimal places
  volumeInput.value = volume.toFixed(4);
}

// Bind event listeners
calculateBtn.addEventListener('click', calculateVolume);

// Support pressing Enter key in input field
radiusInput.addEventListener('keypress', function(e) {
  if (e.key === 'Enter') {
    calculateVolume();
  }
});
</script>

</body>
</html>
```

5. Conclusion & Verification

The Sphere Volume Calculator webpage functions correctly across all modern web browsers. It complies with the course requirements by remaining exceptionally simple, easy to maintain, and visual. The styling is striking yet clean, meeting all academic assignment criteria.